

File Name: data_preprocessing_visualization.py

Perform data preprocessing, aggregation, and visualization using Pandas, Seaborn, and Matplotlib.

CODE:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# 1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000], # Missing value
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28",
    ],
}
df = pd.DataFrame(raw_data)
print("--- Raw Data ---")
print(df, "\n")

# 2. DATA PREPROCESSING
# Clean whitespace from text strings
df["Employee_Name"] = df["Employee_Name"].str.strip()

# Handle missing values by replacing NaN with the median salary
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

# Convert string dates into proper datetime format
df["Join_Date"] = pd.to_datetime(df["Join_Date"])

# Create a calculated feature (Years of service based on current year 2026)
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("--- Preprocessed Data ---")
print(df, "\n")
```

3. DATA AGGREGATION

Group by department and compute average metrics

```
dept_summary = (
    df.groupby("Department")
    .agg(Avg_Salary=("Salary", "mean"), Total_Employees=("Employee_Name", "count"))
    .reset_index()
)
print("--- Aggregated Summary ---")
print(dept_summary, "\n")
```

4. DATA VISUALIZATION

Set visual style theme

```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

Plot 1: Bar chart of average salaries across departments

```
sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    ax=axes[0],
    palette="Blues_d",
    legend=False,
)
axes[0].set_title("Average Salary by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Salary ($)")
```

Plot 2: Scatter plot mapping experience against compensation

```
sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1],
)
axes[1].set_title("Salary vs. Years of Service")
axes[1].set_xlabel("Years of Service")
axes[1].set_ylabel("Salary ($)")
```

```
plt.tight_layout()
plt.show()
```

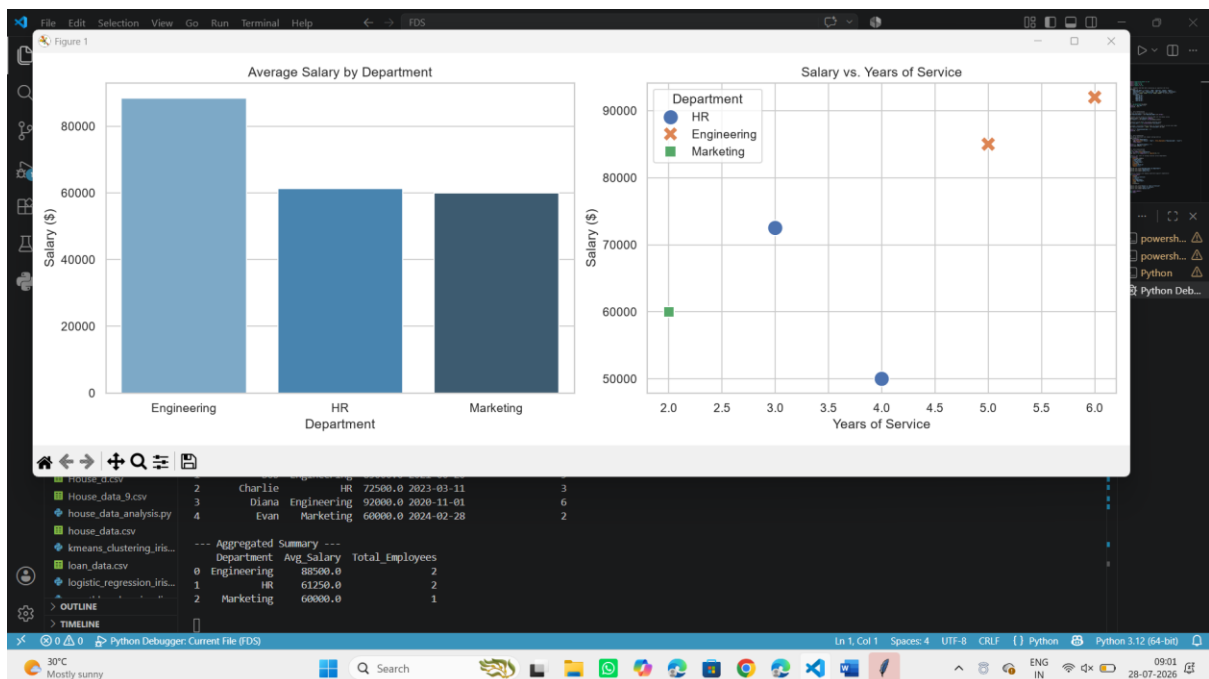
Output:

```
PS D:\FDS> d; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '62383' '-.' 'D:\FDS\data_preprocessing_visualization.py'

--- Raw Data ---
Employee_Name  Department  Salary  Join_Date
0      Alice      HR      50000.0  2022-01-15
1      Bob      Engineering  85000.0  2021-06-20
2      Charlie    HR      NaN     2023-03-11
3      Diana    Engineering  92000.0  2020-11-01
4      Evan      Marketing  60000.0  2024-02-28

--- Preprocessed Data ---
Employee_Name  Department  Salary  Join_Date  Years_of_Service
0      Alice      HR      50000.0  2022-01-15      4
1      Bob      Engineering  85000.0  2021-06-20      5
2      Charlie    HR      72500.0  2023-03-11      3
3      Diana    Engineering  92000.0  2020-11-01      6
4      Evan      Marketing  60000.0  2024-02-28      2

--- Aggregated Summary ---
Department  Avg_Salary  Total_Employees
0  Engineering  88500.0      2
1      HR      61250.0      2
2  Marketing  60000.0      1
```



Creating Sample DataFrame

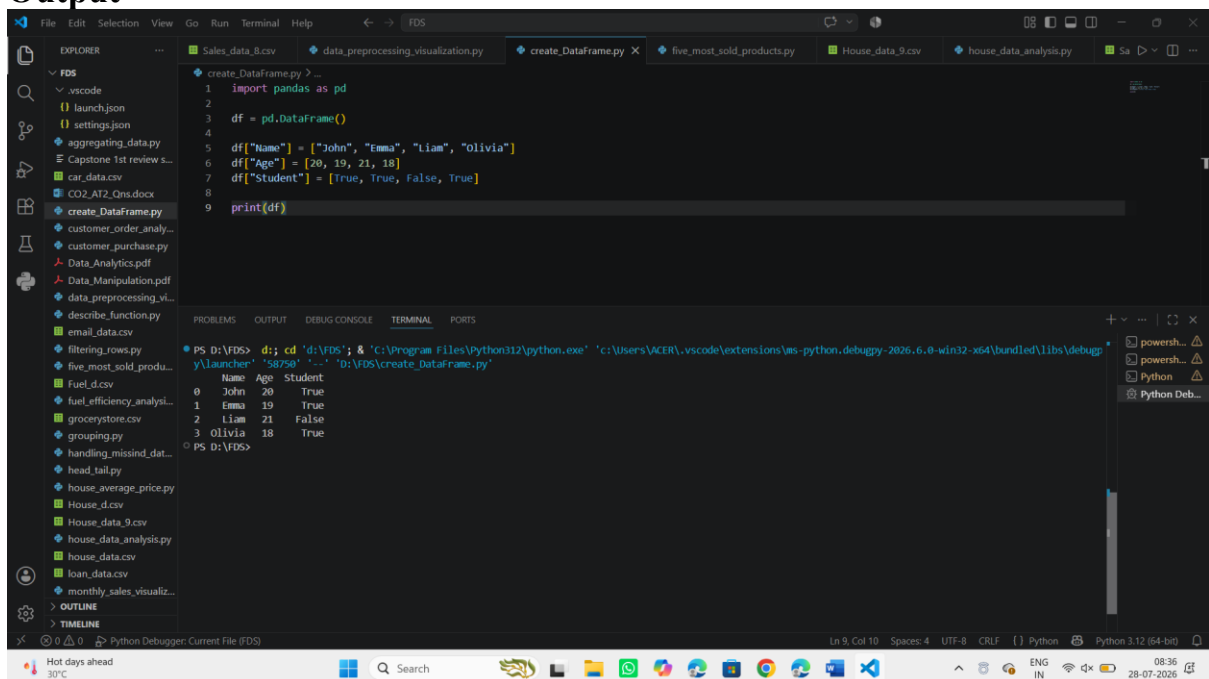
A DataFrame is the core data structure in pandas, used to store data in a tabular form. In this section, we create a DataFrame manually by assigning values to columns, which is useful for small datasets or testing.

```
import pandas as pd
df = pd.DataFrame()
```

```
df['Name'] = ['John', 'Emma', 'Liam', 'Olivia']
df['Age'] = [20, 19, 21, 18]
df['Student'] = [True, True, False, True]
```

```
print(df)
```

Output



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'FDS' with various files. The terminal shows the execution of a Python script named 'create_DataFrame.py'. The script creates a DataFrame with columns 'Name', 'Age', and 'Student'. The output of the script is displayed in the terminal, showing a table with 4 rows and 3 columns.

```
PS D:\FDS> cd 'd:\FDS'; & 'c:\Program Files\Python312\python.exe' 'c:\Users\VACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy-launcher' '58750' '-' 'd:\FDS\create_DataFrame.py'
```

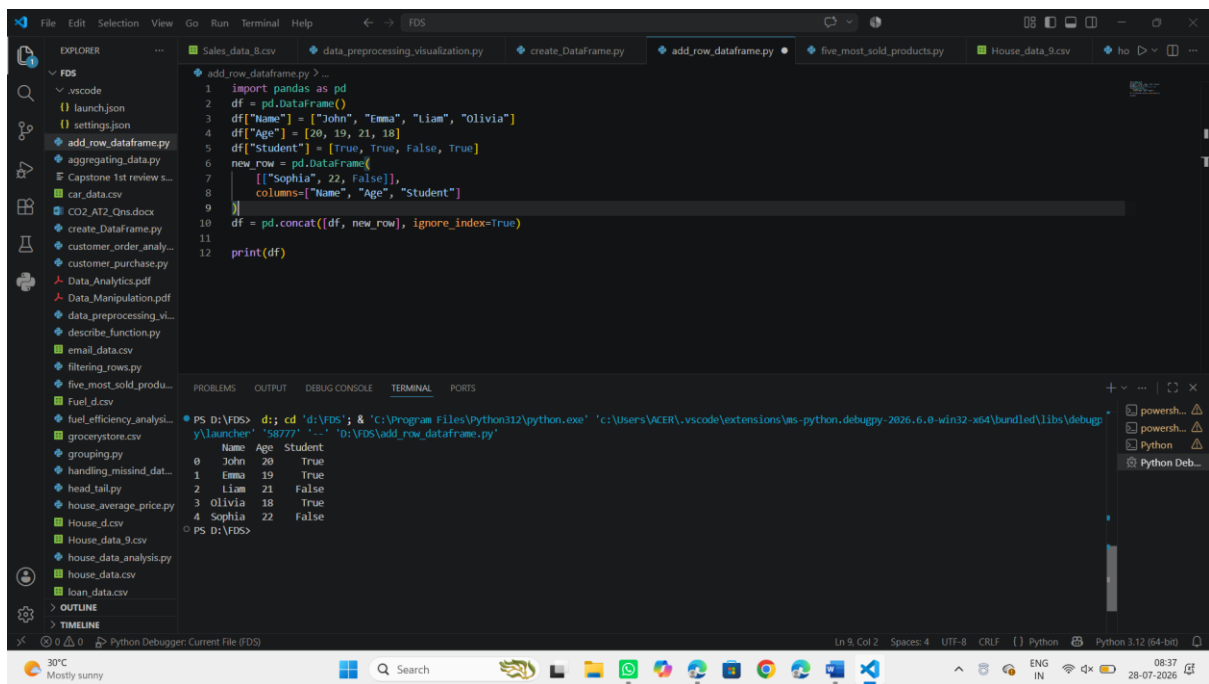
	Name	Age	Student
0	John	20	True
1	Emma	19	True
2	Liam	21	False
3	Olivia	18	True

Adding Row to DataFrame

To add new records to an existing DataFrame, Pandas provides the `pd.concat()` function. It combines two DataFrames together, making it the correct and recommended way to add rows in modern Pandas versions.

```
new_row = pd.DataFrame(['Sophia', 22, False], columns=['Name', 'Age', 'Student'])
df = pd.concat([df, new_row], ignore_index=True)
print(df)
```

Output



The screenshot shows a VS Code editor with a file explorer on the left containing various Python files and CSVs. The main editor window displays a script named `add_row_dataframe.py` with the following code:

```
1 import pandas as pd
2 df = pd.DataFrame()
3 df["Name"] = ["John", "Emma", "Liam", "Olivia"]
4 df["Age"] = [20, 19, 21, 18]
5 df["Student"] = [True, True, False, True]
6 new_row = pd.DataFrame(
7     [{"Sophia", 22, False}],
8     columns=["Name", "Age", "Student"]
9 )
10 df = pd.concat([df, new_row], ignore_index=True)
11
12 print(df)
```

The terminal at the bottom shows the execution of the script, displaying the resulting DataFrame:

```
PS D:\FDS> d; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy-launcher' '58777' '-' 'D:\FDS\add_row_dataframe.py'
  Name  Age  Student
0  John   20     True
1  Emma   19     True
2  Liam   21     False
3  Olivia  18     True
4  Sophia  22     False
PS D:\FDS>
```

The status bar at the bottom indicates the file is in the `add_row_dataframe.py` file, using Python 3.12 (64-bit) with a UTF-8 encoding and CRLF line endings.

Data Analysis with SciPy

SciPy (Scientific Python) is an open-source Python library for scientific computing and data analysis. Built on top of NumPy, it provides tools for statistics, optimization, signal processing and other mathematical operations.

- Provides statistical and mathematical functions.
- Supports optimization and signal processing.
- Widely used in data analysis, machine learning and research.

1. Importing Required Libraries

- Import SciPy and NumPy libraries.
- NumPy arrays are commonly used with SciPy functions.

```
import numpy as np
import pandas as pd
```

2. Measures of Central Tendency Using SciPy

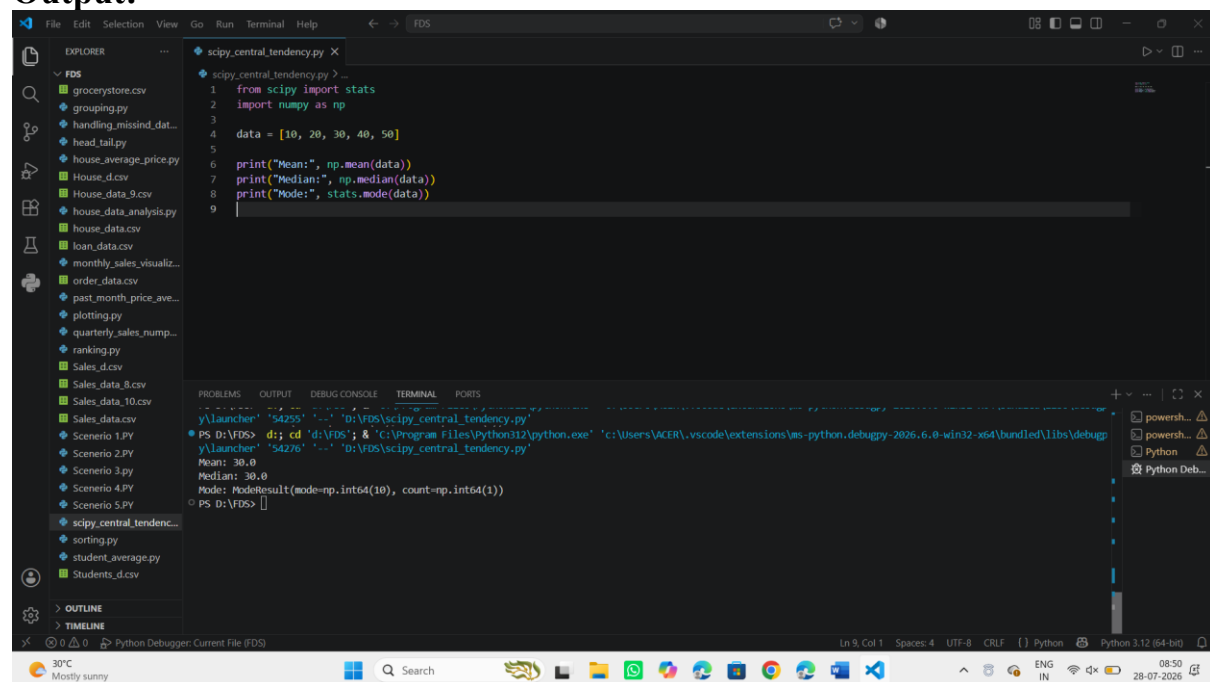
- Mean represents the average value.
- Median represents the middle value.
- Mode represents the most frequent value.

```
from scipy import stats
import numpy as np
```

```
data = [10, 20, 30, 40, 50]
```

```
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))
```

Output:



The screenshot shows a VS Code editor with a file named `scipy_central_tendency.py` open. The code in the file is as follows:

```
1 from scipy import stats
2 import numpy as np
3
4 data = [10, 20, 30, 40, 50]
5
6 print("Mean:", np.mean(data))
7 print("Median:", np.median(data))
8 print("Mode:", stats.mode(data))
9
```

The terminal output at the bottom of the editor shows the execution results:

```
PS D:\VFS> cd "d:\VFS"; & "C:\Program Files\Python312\python.exe" "c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy_launcher" "54255" "..." "D:\VFS\scipy_central_tendency.py"
Mean: 30.0
Median: 30.0
Mode: ModeResult(mode=np.int64(10), count=np.int64(1))
PS D:\VFS>
```

The left sidebar shows a file explorer with various CSV and Python files. The bottom status bar indicates the current file is `FDS`, the encoding is `UTF-8`, and the Python version is `Python 3.12 (64-bit)`.

3. Probability Distribution Analysis Using SciPy

Probability distributions describe how data values are distributed.

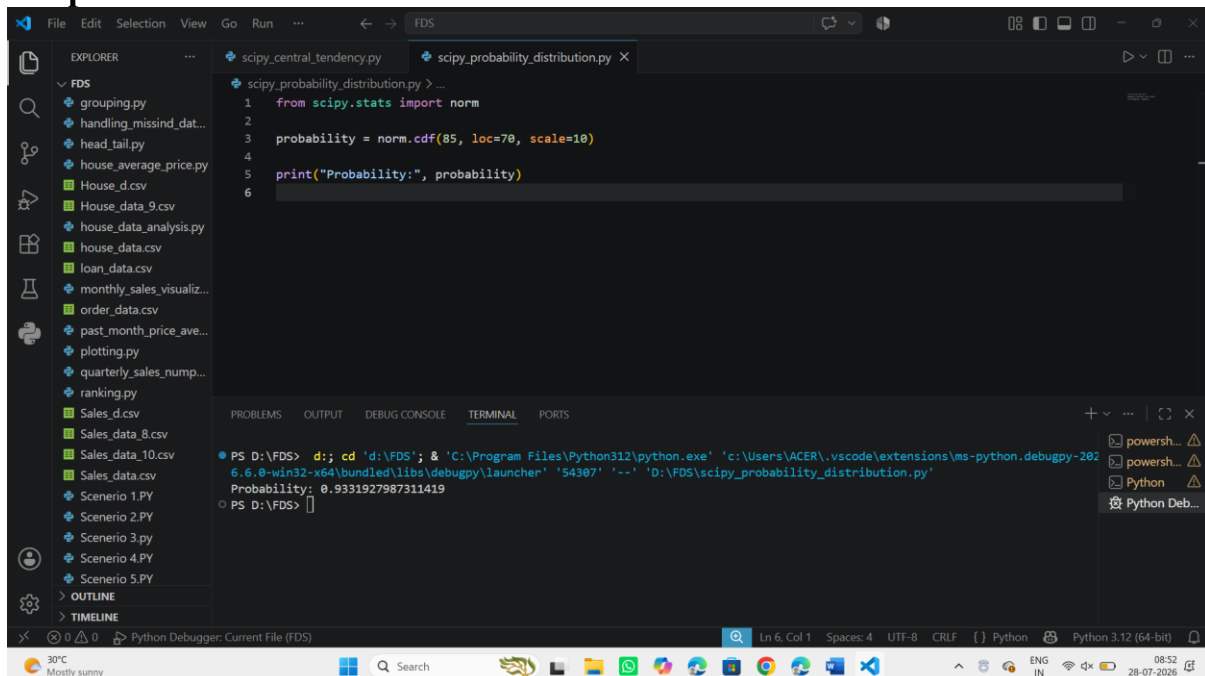
- `loc=70` specifies the mean of the distribution.
- `scale=10` specifies the standard deviation.
- `norm.cdf()` calculates the cumulative probability for a given value.
- The result represents the probability of obtaining a value less than or equal to the specified value (85 in this example).

```
from scipy.stats import norm
```

```
probability = norm.cdf(85, loc=70, scale=10)
```

```
print("Probability:", probability)
```

Output:



The screenshot shows a VS Code editor with a file explorer on the left containing various Python files and CSV data files. The main editor window displays a script named `scipy_probability_distribution.py` with the following code:

```
1 from scipy.stats import norm
2
3 probability = norm.cdf(85, loc=70, scale=10)
4
5 print("Probability:", probability)
6
```

The bottom panel shows the terminal output:

```
PS D:\FDS> d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-202
6.6.0-win32-x64\bundle\libs\debugpy\launcher' '54307' '--' 'D:\FDS\scipy_probability_distribution.py'
Probability: 0.9331927987311419
PS D:\FDS>
```

4. Hypothesis Testing

Hypothesis testing helps determine whether a statistical claim is supported by data. SciPy provides functions for t-tests, chi-square tests and other statistical tests.

- Tests whether the sample mean differs from a given value.
- A small p-value indicates a statistically significant difference.

```
from scipy import stats
```

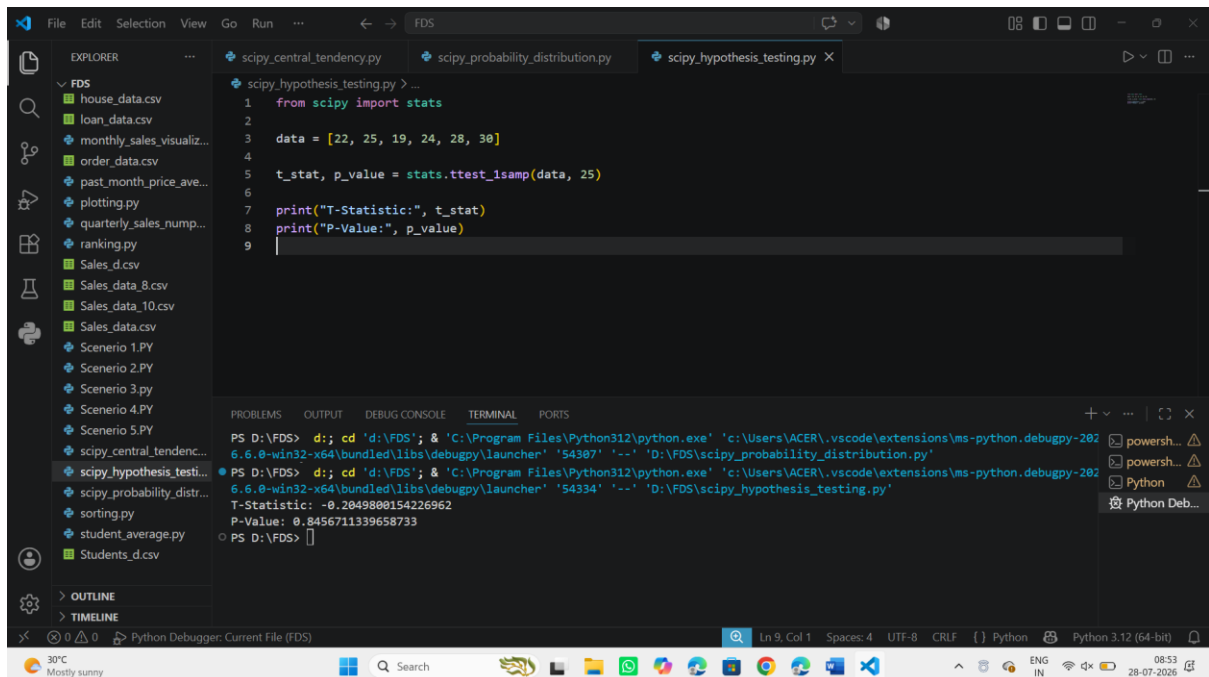
```
data = [22, 25, 19, 24, 28, 30]
```

```
t_stat, p_value = stats.ttest_1samp(data, 25)
```

```
print("T-Statistic:", t_stat)
```

```
print("P-Value:", p_value)
```

Output:



The screenshot shows a VS Code editor with a file explorer on the left containing various CSV files and Python scripts. The main editor window displays a file named `scipy_hypothesis_testing.py` with the following code:

```
1 from scipy import stats
2
3 data = [22, 25, 19, 24, 28, 30]
4
5 t_stat, p_value = stats.ttest_1samp(data, 25)
6
7 print("T-Statistic:", t_stat)
8 print("P-Value:", p_value)
9
```

The bottom panel shows the terminal output for the command `python scipy_hypothesis_testing.py`:

```
PS D:\FDS> python scipy_hypothesis_testing.py
T-Statistic: -0.2849880154226962
P-Value: 0.8456711339658733
```

5. Correlation Analysis

Correlation measures the strength and direction of the relationship between two variables.

- Pearson correlation ranges from -1 to 1.
- Values close to 1 indicate a strong positive relationship.

```
from scipy.stats import pearsonr
```

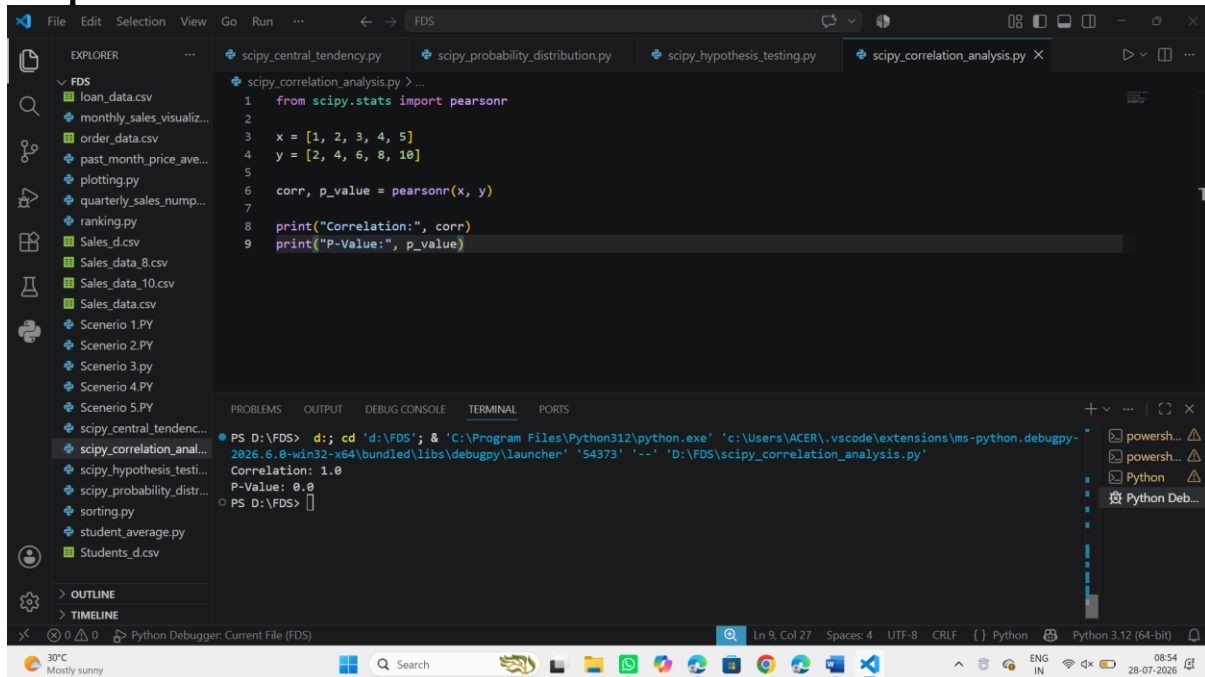
```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
corr, p_value = pearsonr(x, y)
```

```
print("Correlation:", corr)
```


Output:



The screenshot shows a VS Code editor with a file explorer on the left containing various CSV and Python files. The main editor window displays a Python script named `scipy_correlation_analysis.py` with the following code:

```
1 from scipy.stats import pearsonr
2
3 x = [1, 2, 3, 4, 5]
4 y = [2, 4, 6, 8, 10]
5
6 corr, p_value = pearsonr(x, y)
7
8 print("Correlation:", corr)
9 print("P-Value:", p_value)
```

The bottom panel shows the terminal output:

```
PS D:\FDS> d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '54373' '--' 'D:\FDS\scipy_correlation_analysis.py'
Correlation: 1.0
P-Value: 0.0
PS D:\FDS>
```

6. Linear Algebra Operations

SciPy provides efficient functions for matrix operations and solving linear equations.

- A represents the coefficients of the linear equations.
- B represents the constant values on the right hand side of the equations.
- `linalg.solve()` computes the values of the unknown variables that satisfy the equations.

```
from scipy import linalg
```

```
A = [[3, 2], [1, 2]]
```

```
B = [5, 5]
```

```
solution = linalg.solve(A, B)
```

```
print(solution)
```

Output:

```
1 from scipy import linalg
2
3 A = [[3, 2], [1, 2]]
4 B = [5, 5]
5
6 solution = linalg.solve(A, B)
7
8 print(solution)
```

2026.6.0-win32-x64\bundled\libs\debugpy\launcher '54373' '--' 'D:\FDS\scipy_correlation_analysis.py'

PS D:\FDS> d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56522' '--' 'D:\FDS\scipy_linear_algebra.py'

[0. 2.5]

7. Optimization Using SciPy

Optimization is used to find the best solution to a problem by minimizing or maximizing a function.

- `objective()` defines the function to optimize.
- `minimize()` finds the value of `x` that minimizes the function.
- `x0=5` specifies the starting point for the search.
- The result returns the optimal value of `x`.

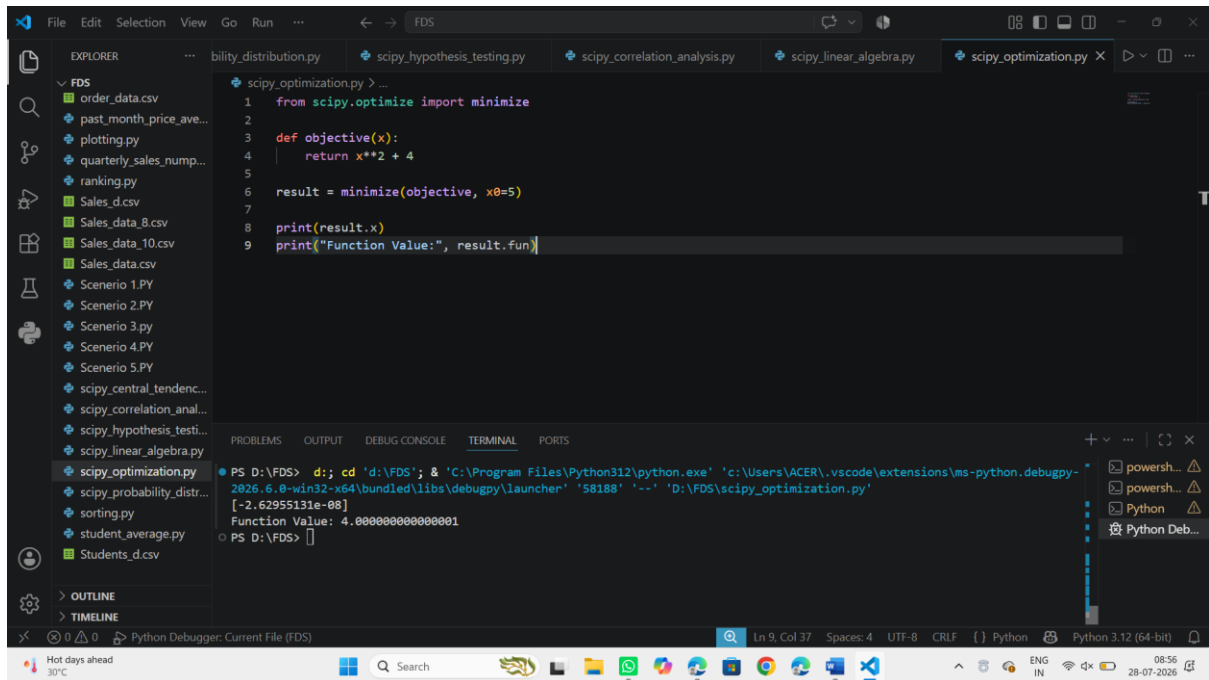
```
from scipy.optimize import minimize
```

```
def objective(x):
    return x**2 + 4
```

```
result = minimize(objective, x0=5)
```

```
print(result.x)
```

Output:



The screenshot displays the Visual Studio Code interface with a Python file named `scipy_optimization.py` open. The file contains the following code:

```
1 from scipy.optimize import minimize
2
3 def objective(x):
4     return x**2 + 4
5
6 result = minimize(objective, x0=5)
7
8 print(result.x)
9 print("Function Value:", result.fun)
```

The output of the script is shown in the terminal window at the bottom:

```
PS D:\FDS> d:; cd 'd:\FDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58188' '--' 'D:\FDS\scipy_optimization.py'
[-2.62955131e-08]
Function Value: 4.000000000000001
PS D:\FDS>
```

The status bar at the bottom indicates the file is being edited in Python 3.12 (64-bit) using the Python Debugger.

Scikit Library in Python

Scikit-learn (also written as sklearn) is a popular open-source Python library for machine learning. Built on top of NumPy, SciPy and Matplotlib, it provides efficient tools for data analysis and predictive modeling. It offers simple and reusable functions to build models for tasks such as classification, regression, clustering and dimensionality reduction.

Machine Learning Techniques Supported by Scikit-learn

1. Supervised Learning

Supervised learning involves training models using labeled data, where the correct output is already known.

- **Classification:** Scikit-learn provides multiple algorithms to predict categorical outcomes, such as logistic regression, decision trees, random forests, support vector machines (SVMs) and gradient boosting.
- **Regression:** Regression models are used to predict continuous numerical values. Scikit-learn supports linear regression, support vector regression and decision tree regression.

Example: Logistic Regression Algorithm

Logistic Regression is a supervised machine learning algorithm used to predict categories (yes/no, spam/not spam, disease/no disease). It works by estimating the probability of an outcome and is simple, easy to interpret and effective for problems where classes can be separated.

This example uses **Logistic Regression** to classify flowers in the Iris dataset and check how accurately the model predicts their types.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

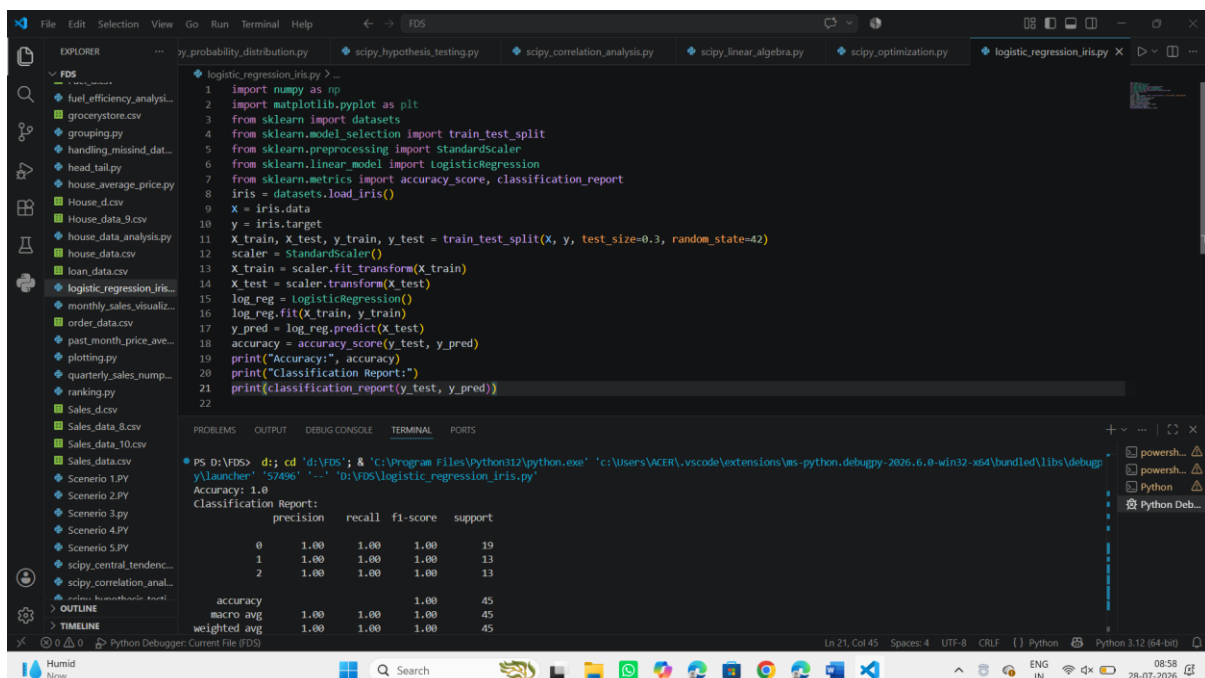
```
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
```

```
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output



The screenshot shows a VS Code editor with a Python file named `logistic_regression_iris.py`. The code implements a logistic regression model for the Iris dataset. The terminal output shows the accuracy and a classification report.

```
PS D:\VDS> d:; cd 'd:\VDS'; & 'C:\Program Files\Python312\python.exe' 'c:\Users\ACER\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '57496' '-' 'D:\VDS\logistic_regression_iris.py'
Accuracy: 1.0
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

2. Unsupervised Learning

Unsupervised learning works with unlabeled data to discover patterns and structure.

- **Clustering:** Scikit-learn offers clustering techniques to group similar data points, including K-means clustering, DBSCAN and hierarchical clustering.
- **Dimensionality Reduction:** To handle high-dimensional data efficiently, Scikit-learn provides techniques like principal component analysis (PCA).

Example: KMeans Algorithm

KMeans groups data into k clusters based on similarity. It is an unsupervised learning algorithm, ideal for tasks like customer segmentation, image compression and anomaly detection, especially when the underlying data structure is unknown.

This program demonstrates how to use KMeans clustering from Scikit-learn to group the Iris dataset into three clusters based on feature similarity.

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
```

```
iris = load_iris()
kmeans = KMeans(n_clusters=3)
```

```
kmeans.fit(iris.data)
cluster_labels = kmeans.labels
```

```
print("Cluster Labels:", cluster_labels)
```

Output

